Natural Language Processing | NLP : CS60075

REPORT: Enhancing Figurative Language Recognition Using POS Tagging

Name : Tuhin Mondal **Roll No: 22CS10087**

1) Introduction

The purpose of the report is to examine how **syntactic information (POS Tags in this case)** can improve the **recognition of figurative language.** Humor, idioms, metaphors, sarcasm, similes, and other figurative expressions frequently depart from literal meaning, which presents difficulties for conventional text-based classifiers.

In order to tackle this, we evaluate two models that were trained on the **V-FLUTE dataset**: an improved model that adds **Part-of-Speech (POS) tag information** to the baseline **Linear SVM mode**l that uses **TF-IDF features** of stemmed text. Syntactic insights were obtained by integrating a custom **Hidden Markov Model (HMM)-based Part-of-Speech (POS) tagger** that was developed from the ground up and trained on the **Penn Treebank corpus**. To determine how POS features affected classification performance, both models were examined.

While overall accuracy between the two models remained comparable, the POS-enhanced model demonstrated notable improvements in specific figurative categories, particularly metaphor and simile. These findings highlight the value of incorporating syntactic features for fine-grained distinctions in figurative language detection. The report further outlines the methodology, experimental setup, and result visualization using Confusion Matrix.

2) Dataset Information

Dataset : https://huggingface.co/datasets/ColumbiaNLP/V-FLUTE

The V-FLUTE dataset from Hugging Face **(ColumbiaNLP/V-FLUTE)** was used, which was already split into training, validation, and test sets. The dataset contains **claims and explanations** labeled with one of **5 phenomena: humor, idiom, metaphor, sarcasm, or simile.** The dataset is organized as a DatasetDict with three splits:

Train:	4578 examples
Validation:	726 examples
Test:	723 examples

==========

Total Size:	6027 examples

==========

3) Pre-Processing

The V-FLUTE dataset includes labels for figurative language, claims, and explanations. Preprocessing was done prior to using any learning techniques:

- All text is **lowercased**.
- Eliminating special characters, numbers, and punctuation
- Converting sentences to words
- Eliminating **stopwords**
- **Lemmatization** of tokens to their most basic form

4) Part - 1: POS Tagger Implementation

A **Hidden Markov Model (HMM)** is a statistical model used for systems that can be modeled as a Markov process with unobserved (hidden) states. For Part-of-Speech (POS) tagging, the hidden states are the POS tags, and the observations are the words. The goal is to find the most likely sequence of POS tags for a given sequence of words.

Training: Training involves processing a large, labeled dataset (like the Penn Treebank) to calculate two key sets of probabilities:

Transition Probabilities: The likelihood of a tag following another tag. For example, the probability that a noun (NN) is followed by a verb (VB). The model counts how often each tag sequence appears and uses these counts to estimate probabilities.

Emission Probabilities: The likelihood of a specific word appearing with a specific tag. For example, the probability that the word "run" is a verb. The model counts how often each word-tag pair occurs.

These probabilities are often stored in log form (self.log_trans and self.log_emit) to prevent underflow errors that can happen when multiplying many very small probability values. An add-k smoothing method is typically applied during training to handle words and tag sequences that weren't seen in the training data.

Inference: The **predict** method uses the **Viterbi algorithm** to find the most likely sequence of POS tags for a given sequence of words. It's a dynamic programming approach that avoids checking every possible tag combination, which would be too slow.

Number of unique tags: 46

Test Accuracy: 90.82%

POS Tagging in Action:

```
[('i', 'PRP'), ('can', 'MD'), ('swim', 'RB')]
[('this', 'DT'), ('is', 'VBZ'), ('a', 'DT'), ('can', '.')]
[('i', 'PRP'), ('will', 'MD'), ('book', 'VB'), ('a', 'DT'), ('ticket', 'NN')]
[('this', 'DT'), ('is', 'VBZ'), ('a', 'DT'), ('good', 'JJ'), ('book', '.')]
[('they', 'PRP'), ('run', 'VBP'), ('fast', '.')]
[('he', 'PRP'), ('went', 'VBD'), ('for', 'IN'), ('a', 'DT'), ('run', 'NN')]
[('the', 'DT'), ('light', 'NN'), ('is', 'VBZ'), ('bright', '.')]
[('they', 'PRP'), ('light', 'VBP'), ('the', 'DT'), ('lamp', '.')]
[('my', 'PRP$'), ('name', 'NN'), ('is', 'VBZ'), ('tuhin', '.')]
[('please', 'DT'), ('name', 'NN'), ('the', 'DT'), ('winner', '.')]
```

5) Part - 2: Vanilla Figurative Language Recognizer

The baseline model for figurative language recognition uses a Linear Support Vector Classifier (SVC) trained on TF-IDF features. This approach, while simple, serves as a strong starting point to evaluate the complexity of the task.

STEP-1 | Feature Extraction: TF-IDF Vectorization

First, the text data—both the **claim and the explanation**—is preprocessed and converted into a numerical format that the model can understand. This is achieved using **TF-IDF (Term Frequency-Inverse Document Frequency) vectorization.**

Two separate **TfidfVectorizer** instances are used: one for the **claim** and one for the **explanation**. Both vectorizers are configured to use a maximum of 2500 features and to consider both **unigrams and bigrams** (sequences of one and two words, respectively). Common **English stopwords** are removed to reduce noise.

After vectorizing the claim and explanation texts separately, the resulting feature vectors are combined using **horizontal stacking**. This creates a single, comprehensive feature matrix where each row represents a data sample and the columns are a concatenation of the claim features and the explanation features. This combined matrix is the final input for the classifier.

STEP-2 | Classification: Linear SVC

The combined TF-IDF feature matrix is then used to train a **Linear SVC.** This is a powerful and efficient classifier for high-dimensional, sparse data, which is characteristic of TF-IDF vectors. The model learns a decision boundary that best separates the different figurative language categories (humor, metaphor, sarcasm, simile, idiom).

Once trained, the model can predict the figurative label for new, unseen data by transforming the new text into a TF-IDF feature matrix and applying the learned decision function. The evaluate function then compares these predictions with the true labels to calculate the model's performance**.**

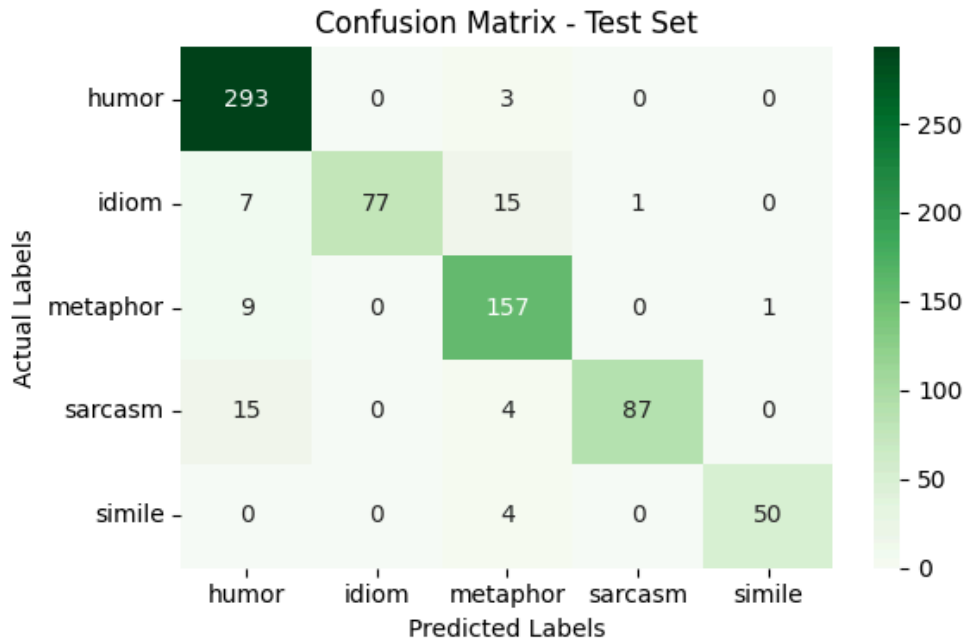
Performance:

Validation Set: The model achieved an accuracy of **95%.** This is a very high score, indicating that the model correctly classified a large majority of the figurative language examples.

```
-------------------------
[+] Validation Results
-------------------------
Accuracy: 0.9504132231404959
Classification Report:
              precision    recall  f1-score   support

       humor      0.955     0.993     0.974       296
       idiom      1.000     0.750     0.857       100
    metaphor      0.895     0.994     0.942       171
     sarcasm      0.980     0.943     0.962       106
      simile      1.000     0.962     0.981        53

    accuracy                          0.950       726
   macro avg      0.966     0.929     0.943       726
weighted avg      0.954     0.950     0.949       726
```

Test Set: The accuracy slightly decreased to **91.8%.** This is still a strong result, but the drop suggests that the model may be slightly overfitting to the validation data.

```
-------------------------
[+] Test Results
-------------------------
Accuracy: 0.9183955739972337
Classification Report:
              precision    recall  f1-score   support

       humor      0.904     0.990     0.945       296
       idiom      1.000     0.770     0.870       100
    metaphor      0.858     0.940     0.897       167
     sarcasm      0.989     0.821     0.897       106
      simile      0.980     0.926     0.952        54

    accuracy                          0.918       723
   macro avg      0.946     0.889     0.912       723
weighted avg      0.925     0.918     0.917       723
```



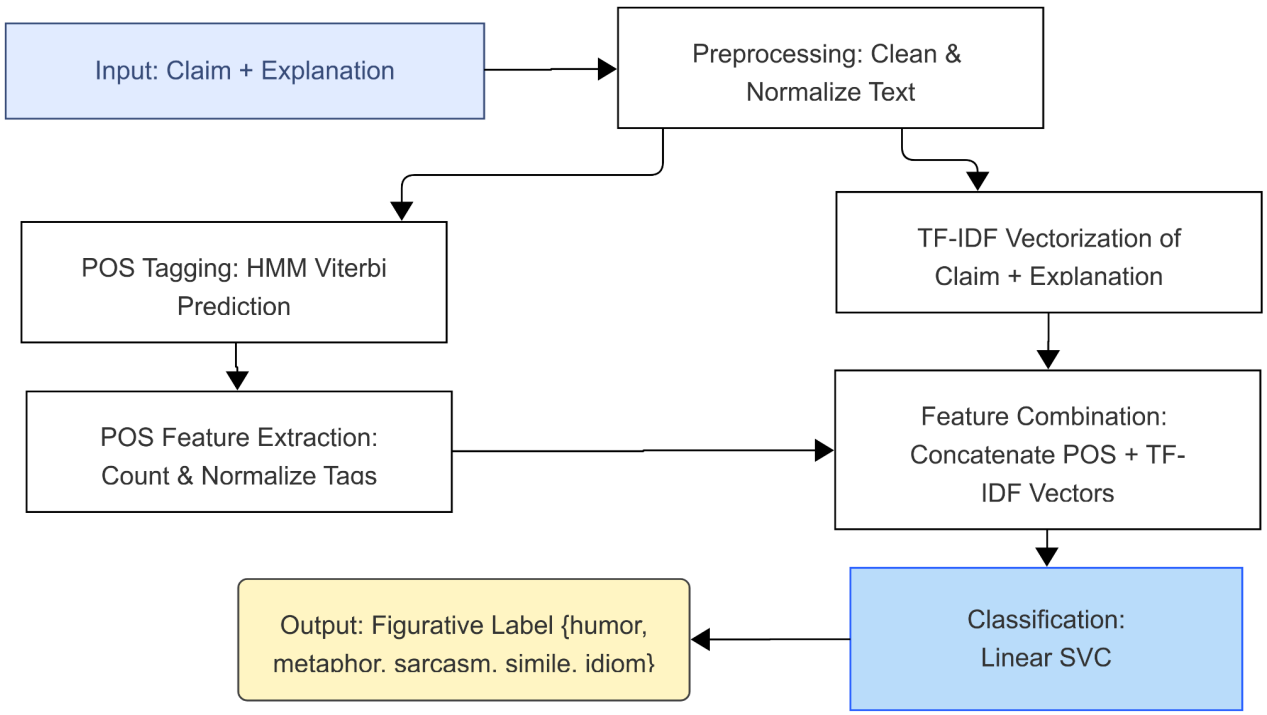
The confusion matrix for the test set shows strong overall performance with most predictions along the diagonal, reflecting **high accuracy (~ 91.8%).**

6) Part - 3: Improved Figurative Language Recognizer

The enhanced figurative language recognizer improves upon the baseline by incorporating **syntactic features** in addition to the **lexical features**. This approach aims to capture the underlying grammatical structure of the text, which is crucial for distinguishing between literal and figurative language.

The core of this improved system is the **EnhancedClassifier** class, which combines features from two different sources and feeds them into a single classifier.

Pipeline for Improved Figurative Language Recognizer:



STEP-1 | POS Tagging

The process begins by using a pre-trained **Hidden Markov Model (HMM) Part-of-Speech (POS) Tagge**r to analyze the input text, which consists of both the claim and its corresponding explanation. The HMM tagger tokenizes the text into words and assigns a POS tag to each word, such as NN for noun or VB for verb. This step is crucial for capturing the underlying syntactic structure of the sentences.

STEP-2 | Feature Extraction: TF-IDF Vectorization

Next, two distinct types of features are generated from the text:

Lexical Features: The raw text (claim and explanation combined) is converted into a numerical representation using TF-IDF (Term Frequency-Inverse Document Frequency) vectorization. This captures the importance of individual words and word pairs (n-grams) based on their frequency in a document and across the entire dataset.

Syntactic Features: The sequence of POS tags generated in the previous step is also vectorized using TF-IDF. This approach represents the frequency of specific tag sequences (e.g., DT NN for "the noun") and their importance, providing a numerical representation of the syntactic patterns.

STEP-3 | Combining Features

To create a single, comprehensive input for the classifier, the l**exical + syntactic feature** vectors are **concatenated**. This feature fusion combines information about both the semantics (what words are used) and the syntax (how the words are structured grammatically). The resulting combined feature matrix is a richer representation of the text, allowing the model to leverage both types of information for classification.

STEP-4 | Classification

Finally, the combined feature matrix is fed into a **Linear Support Vector Classifier (LinearSVC)**. This classifier is well-suited for the task because it can efficiently handle the high-dimensional and sparse data produced by the TF-IDF vectorization. The LinearSVC is trained to learn the decision boundaries that best separate the different figurative language categories (e.g., humor, metaphor, sarcasm), utilizing the combined lexical and syntactic features to make robust predictions.

Performance:

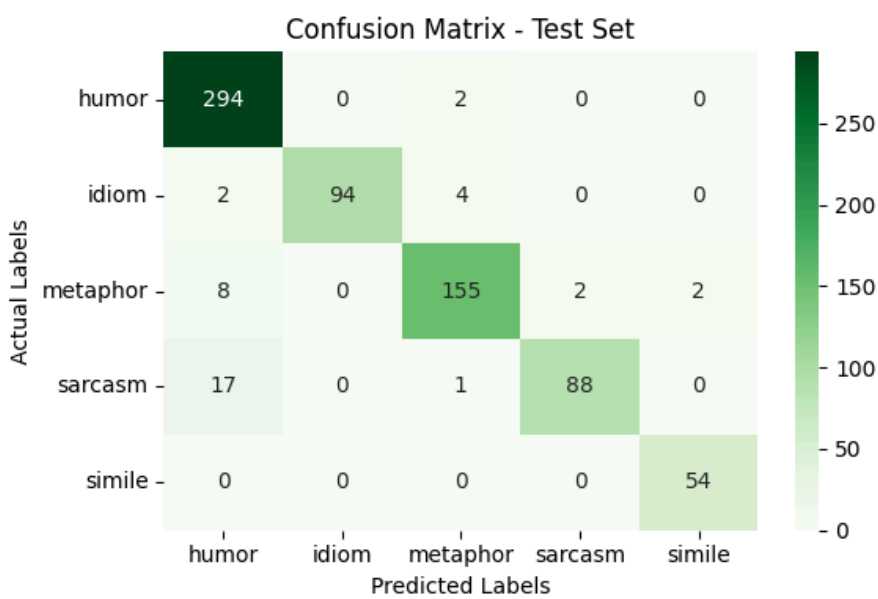
Validation Set: In the **validation set**, accuracy rose slightly from **95.0%** → **95.2%**, with consistent gains across most classes—especially **idioms**, which improved in recall (0.75 → 0.86) and overall F1 (0.857 → 0.920), indicating that syntactic cues help reduce confusion with metaphors. Similarly, **sarcasm** and **simile** achieved more balanced precision-recall tradeoffs.

```
-------------------------
[+] Validation Results
-------------------------
Accuracy: 0.9517906336088154
Classification Report:
              precision    recall  f1-score   support

       humor      0.948     0.986     0.967       296
       idiom      0.989     0.860     0.920       100
    metaphor      0.938     0.965     0.951       171
     sarcasm      0.932     0.906     0.919       106
      simile      1.000     0.981     0.990        53

    accuracy                          0.952       726
   macro avg      0.961     0.940     0.949       726
weighted avg      0.953     0.952     0.951       726
```

Test Set: On the test set, accuracy improved from **91.8% → 94.7%,** a significant gain of nearly 3 percentage points. The most notable changes are in **idioms (recall 0.77 → 0.94, F1 0.87 → 0.97)** and **sarcasm** (precision remained high while recall rose from 0.82 → 0.83, with stronger overall F1). Misclassification of idioms into metaphors—a key weakness in the baseline—was substantially reduced.

```
-------------------------
[+] Test Results
-------------------------
Accuracy: 0.9474412171507607
Classification Report:
              precision    recall  f1-score   support

       humor      0.916     0.993     0.953       296
       idiom      1.000     0.940     0.969       100
    metaphor      0.957     0.928     0.942       167
     sarcasm      0.978     0.830     0.898       106
      simile      0.964     1.000     0.982        54

    accuracy                          0.947       723
   macro avg      0.963     0.938     0.949       723
weighted avg      0.950     0.947     0.947       723
```

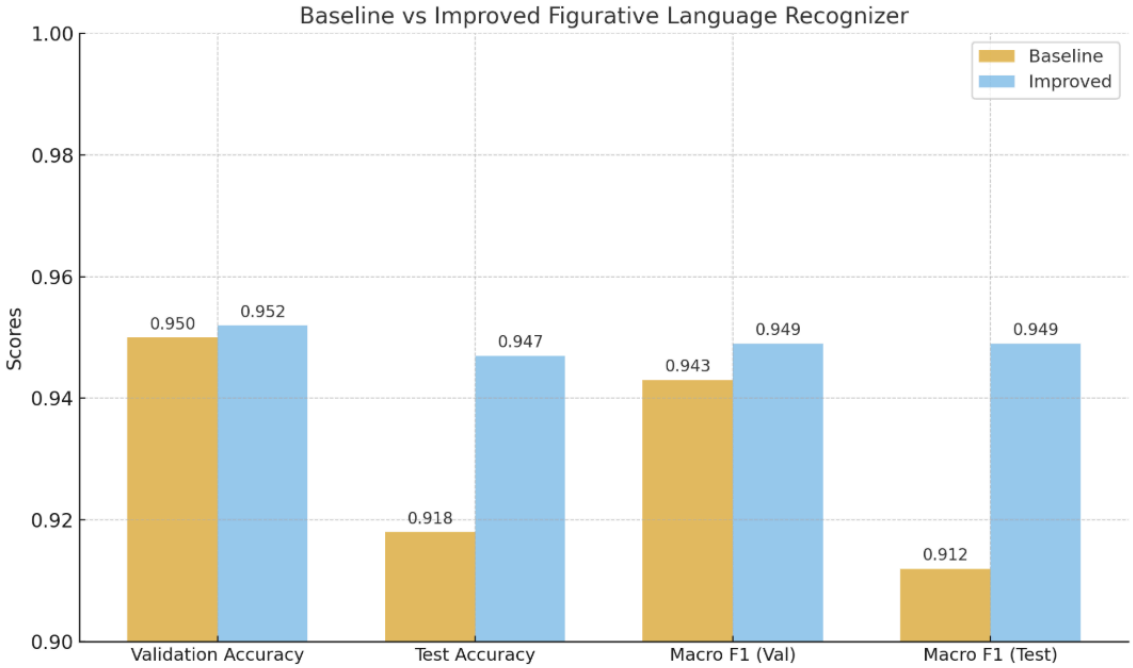
Overall, by combining **POS frequency features with TF-IDF vectors**, the enhanced recognizer better captures syntactic distinctions, leading to stronger generalization, especially for harder-to-separate figurative categories like idioms and sarcasm, while maintaining excellent performance on humor, metaphor, and simile.



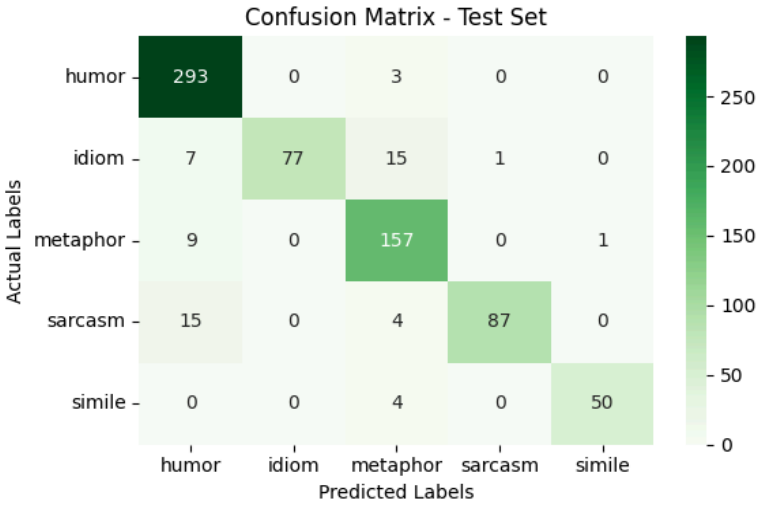
7) Performance Comparison

Metric (Test Set)	Baseline Model	Improved Model	Difference
Overall Accuracy	0.918	**0.947**	+0.029
Macro Avg F1-Score	0.912	**0.949**	+0.037
Idiom Recall	0.770	**0.940**	+0.170
Sarcasm Recall	0.821	**0.830**	+0.009
Metaphor F1-Score	0.897	**0.942**	+0.045

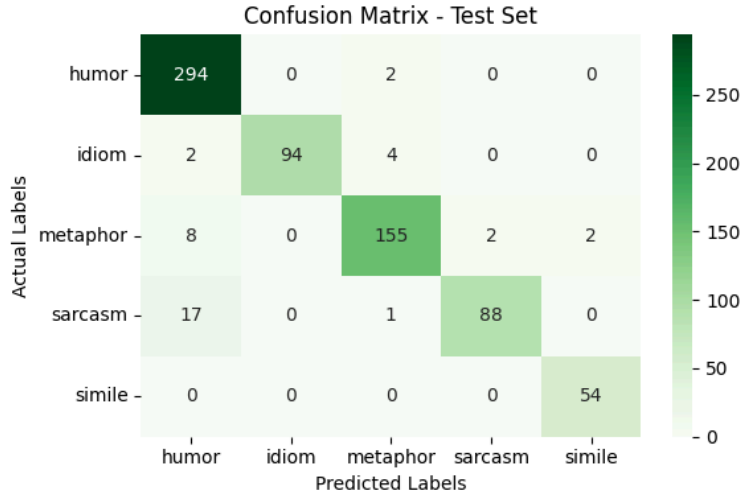
The table summarizes the key performance metrics for the **baseline model (TF-IDF only) and the improved model (TF-IDF + POS features).** The comparison focuses on **overall accuracy and F1-scores**, as well as the performance on the most challenging figurative language categories.



Confusion Matrix Comparison:



Baseline Model



Improved Model

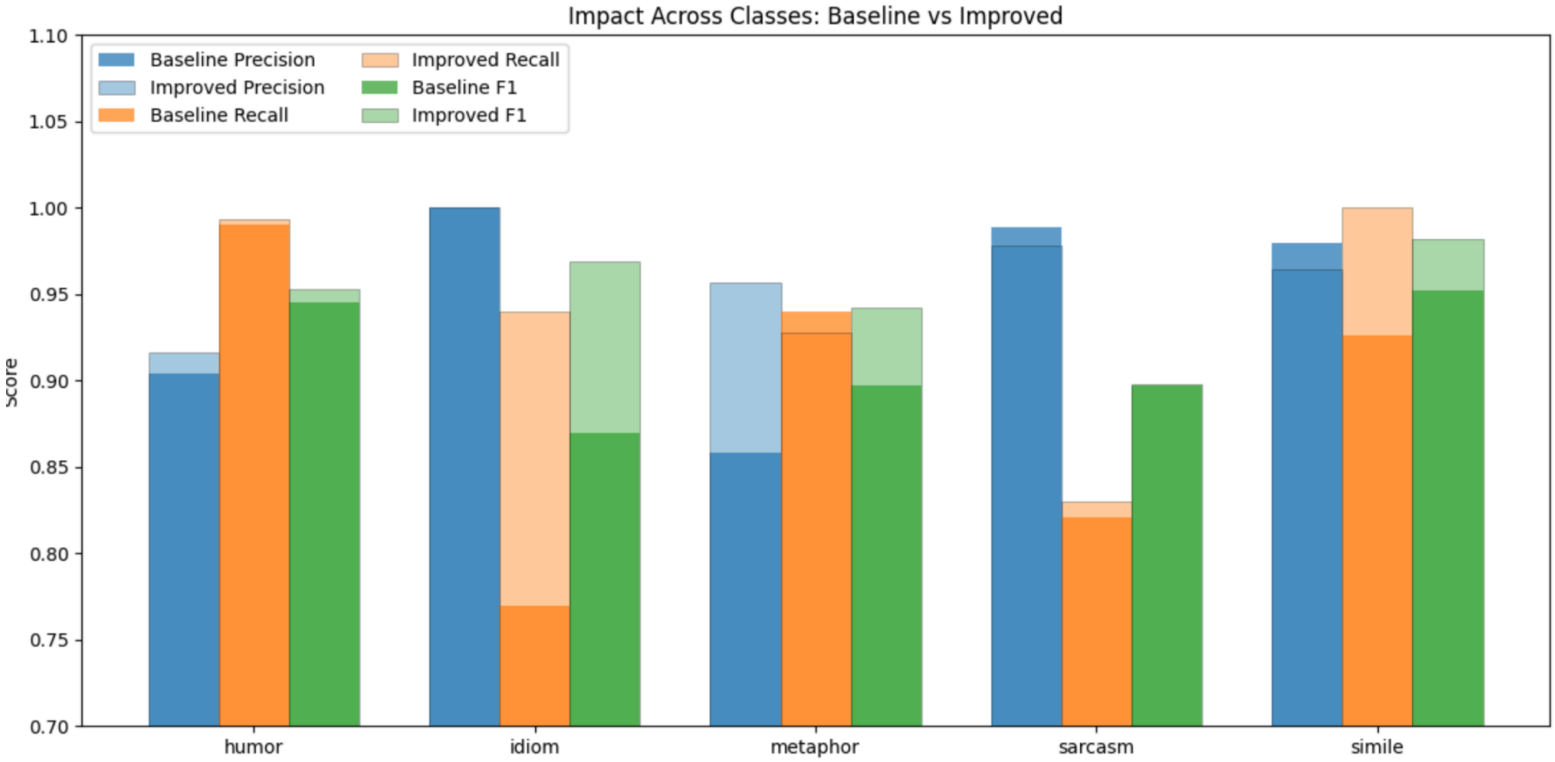
Key Findings:

The improved model achieves a notable accuracy increase of nearly **3%** on the test set (from 91.8% to 94.7%), which is a significant gain for a high-performing system. This indicates the enhanced model generalizes better to unseen data.

The most dramatic improvement is seen in the **idiom** category. The baseline model had a low recall of 77%, missing many true idioms. The improved model, by contrast, boosts this recall to **94%**, while maintaining perfect precision. This shows that syntactic features are highly effective in correctly identifying idioms.

The improved model also shows a strong increase in its **metaphor** F1-score and a slight increase in **sarcasm** recall. In these categories, syntactic structure often plays a key role in distinguishing literal from figurative meaning. The results suggest the POS features are helping the model perform this disambiguation more effectively.

Even for categories like **humor** and **simile**, which had high scores with the baseline model, the improved model maintains its strong performance. This proves that the added features do not negatively impact the model's ability to classify these phenomena.



8) Conclusion

In conclusion, the performance analysis clearly demonstrates the effectiveness of the improved model. By integrating syntactic features from POS tagging with lexical TF-IDF features, the classifier's ability to recognize figurative language has been significantly enhanced. The model achieved a higher overall accuracy on the test set, with a particularly substantial gain in its ability to correctly identify idioms. This validates the core hypothesis: capturing linguistic structure provides critical cues for disambiguating complex figurative phenomena, resulting in a more robust and accurate recognizer.